
Dossier de conception technique & architecture

Jalon 4 – Projet Fil Rouge CDA

Étudiant FORTUNATO Axel

Formation IPSSI — CDA Bachelor Fullstack DevOps

Promotion 2025–2026

Date Avril 2026

Table des matières

1. Objectif du jalon 4	2
2. Diagrammes UML – Cas d'utilisation (Use Cases)	3
2.1 Acteurs	5
3. Diagrammes UML – Séquences (2 à 3 scénarios principaux)	6
3.1 Séquence 1 – Lecture d'une page et sauvegarde de progression	6
3.2 Séquence 2 – Ajouter / retirer un favori	6
3.3 Séquence 3 – Lecture audio (TTS) d'une page	7
4. Diagramme de classes UML (Back-end)	8
5. Architecture multi-couches	9
5.1 Pattern MVC (implémentation Symfony)	9
5.2 Architecture n-tiers (vue simple)	9
5.3 Séparation des responsabilités et bonnes pratiques	9
5.4 Composants externes / bibliothèques	10
6. Schémas complémentaires (optionnel)	11
6.1 États de publication d'une histoire	11
7. Conclusion du jalon 4	11

1. Objectif du jalon 4

Ce document formalise la **conception technique** de l'application de lecture d'histoires pour enfants, en cohérence avec :

- le **CDCF** (Jalon 1) : périmètre fonctionnel et exigences,
- la **conception UI/UX** (Jalon 2) : écrans et parcours utilisateur,
- la **modélisation BDD** (Jalon 3) : MCD/MLD/MPD (entités User, Story, Page, Favorite, ReadingProgress).

Il fournit :

- les **diagrammes UML** (cas d'utilisation, séquences, classes),
 - la **description d'architecture multi-couches / n-tiers**,
 - les choix d'implémentation et l'intégration des composants externes (Text-to-Speech).
-

2. Diagrammes UML – Cas d'utilisation (Use Cases)

Les cas d'utilisation ci-dessous couvrent l'ensemble des exigences fonctionnelles du CDCF (gestion utilisateurs, catalogue, lecture paginée, favoris/progression, administration, lecture audio).

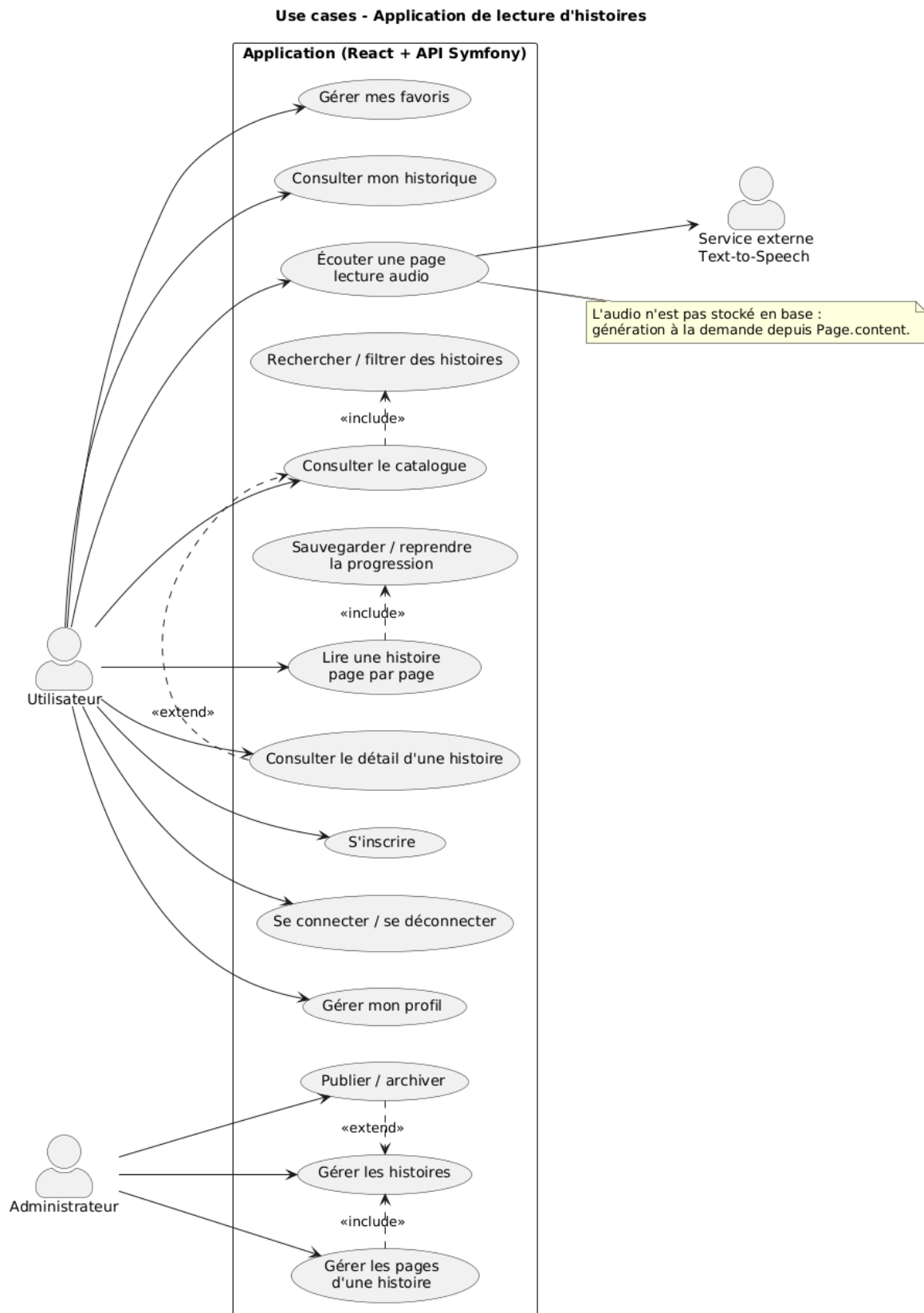


Fig. 1 : Diagramme UML – Cas d'utilisation

2.1 Acteurs

- **Utilisateur** : consulte le catalogue, lit les histoires, gère favoris et progression, utilise la lecture audio.
 - **Administrateur** : gère/modère les contenus (histoires et pages), publie/archivage.
 - **Service externe TTS** : API de synthèse vocale (acteur externe, appelé par le back-end).
-

3. Diagrammes UML – Séquences (2 à 3 scénarios principaux)

Les diagrammes suivants détaillent l'enchaînement des messages entre front-end React, API Symfony (controllers/services), Doctrine (repositories/entités) et la base MySQL.

3.1 Séquence 1 – Lecture d'une page et sauvegarde de progression

Objectif : afficher une page, puis enregistrer la progression (reprise).

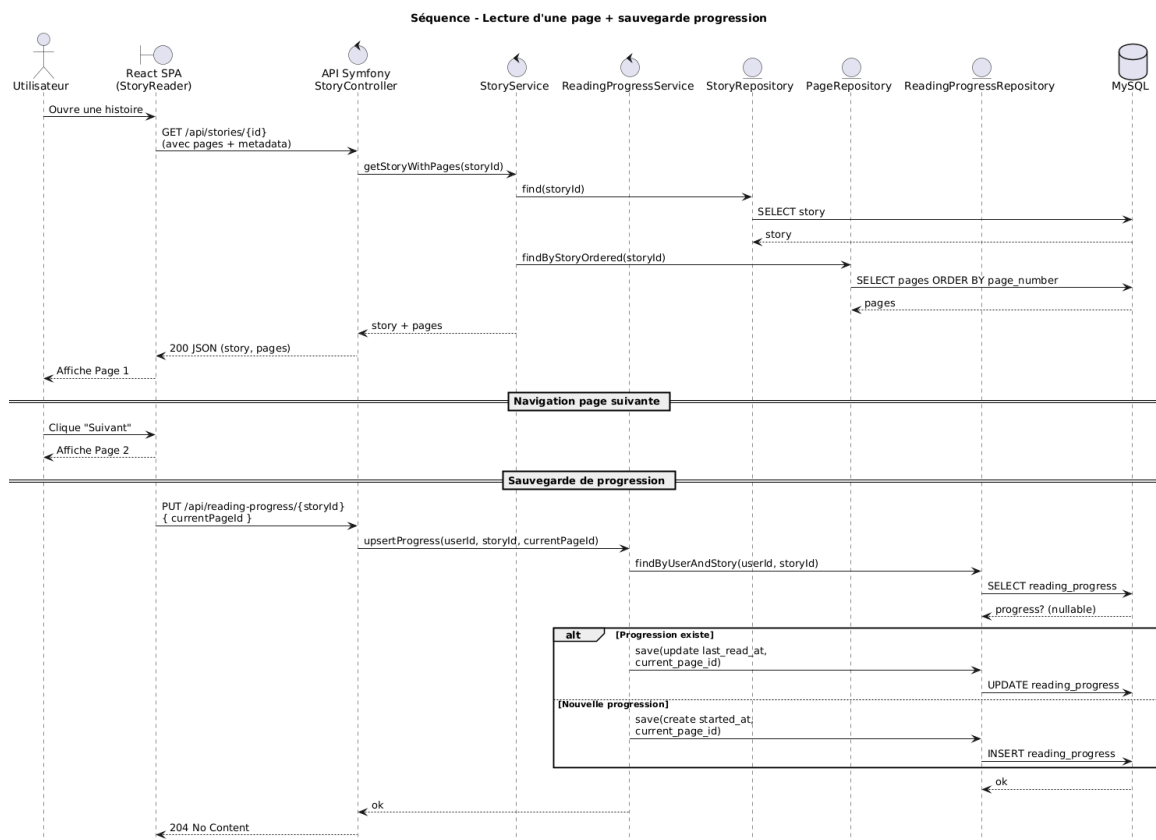


Fig. 2 : Diagramme UML – Séquence lecture + progression

3.2 Séquence 2 – Ajouter / retirer un favori

Objectif : toggler un favori depuis le catalogue ou la page d'histoire.

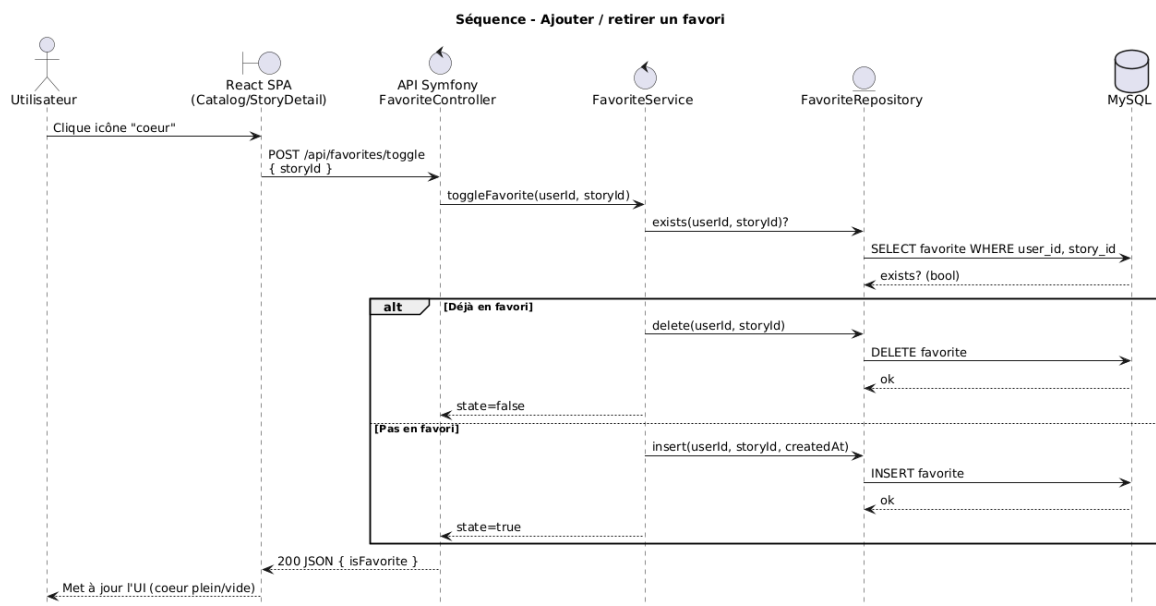


Fig. 3 : Diagramme UML – Séquence toggle favori

3.3 Séquence 3 – Lecture audio (TTS) d'une page

Objectif : demander la synthèse vocale pour une page (sans stocker d'audio en base).

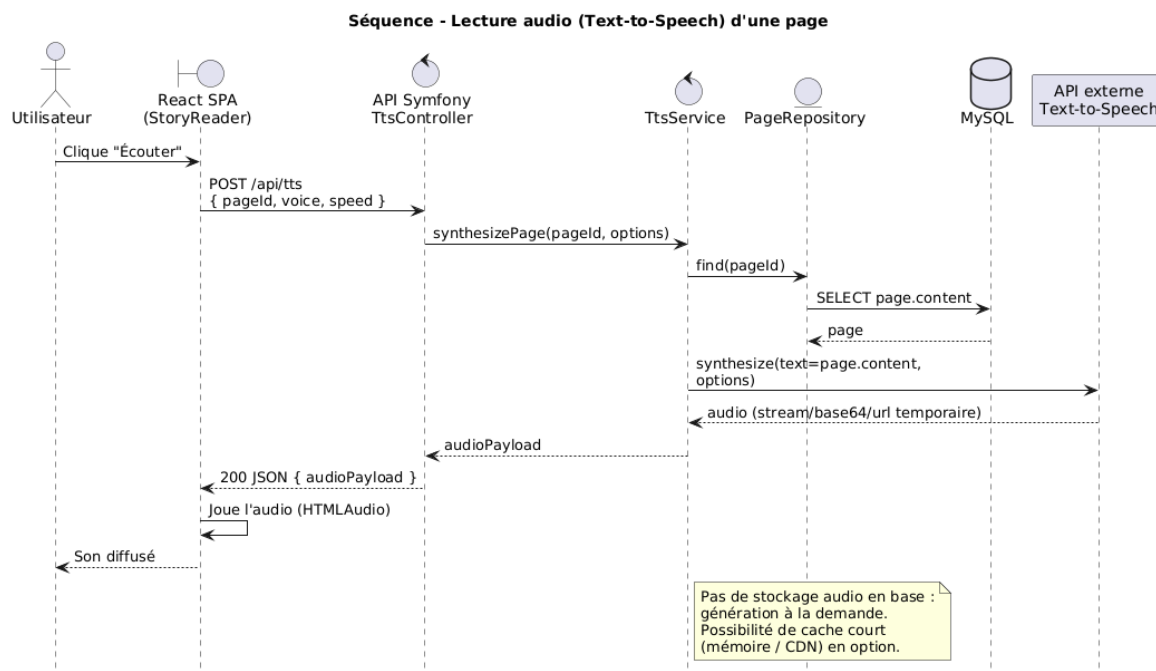


Fig. 4 : Diagramme UML – Séquence lecture audio (TTS)

4. Diagramme de classes UML (Back-end)

Le diagramme ci-dessous se concentre sur :

- les **entités** issues de la BDD (Jalon 3),
- les **repositories** Doctrine,
- la **couche service** (logique métier),
- les **controllers** (API REST).

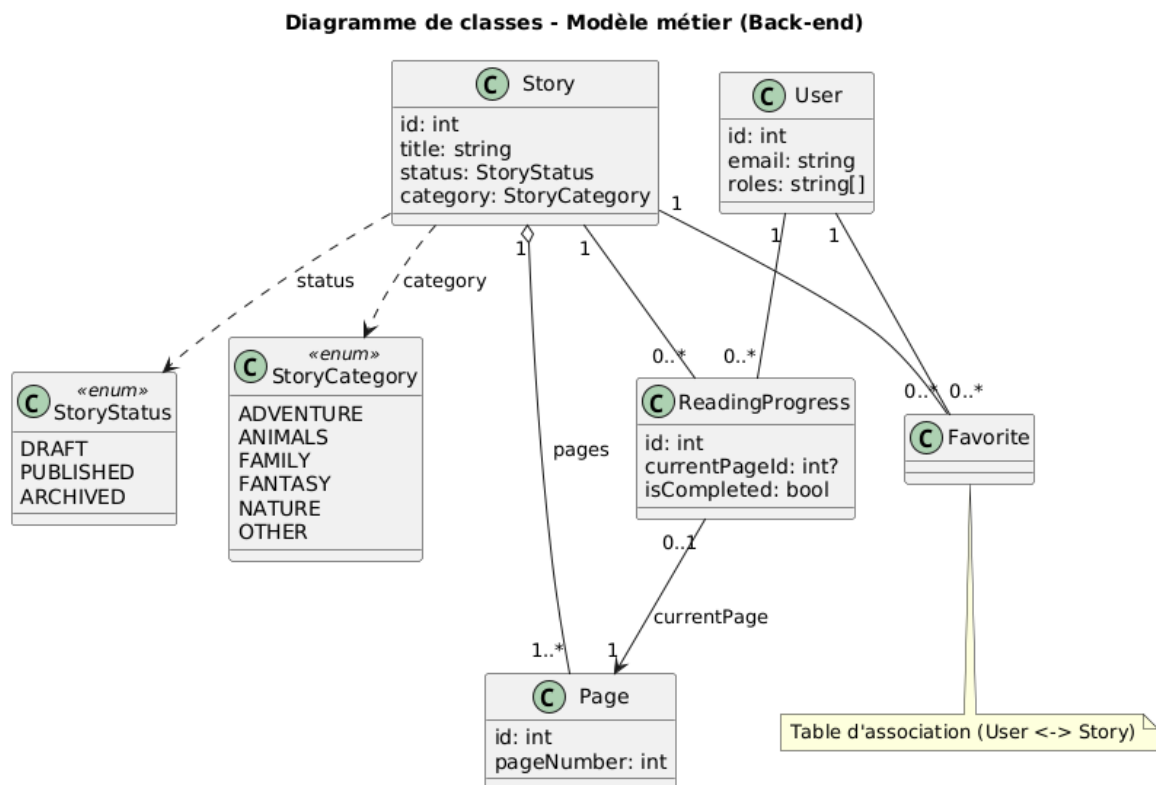


Fig. 5 : Diagramme UML – Classes (Back-end)

5. Architecture multi-couches

5.1 Pattern MVC (implémentation Symfony)

Même si l'application est une API REST (sans vues Twig), l'organisation Symfony reste alignée avec l'esprit MVC :

- **Controllers** : reçoivent les requêtes HTTP, valident les entrées (DTO/constraints), orchestrent l'appel aux services, renvoient une réponse JSON.
- **Services (métier)** : portent la logique applicative (règles de progression, règles favoris, publication, etc.).
- **Model / Accès données** : entités Doctrine + repositories + migrations.

Le **front-end React (SPA)** constitue la couche présentation (équivalent "View" dans une archi séparée).

5.2 Architecture n-tiers (vue simple)

Architecture logique 3-tiers (niveau "compréhensible par tous") :

- **Tier 1 – Client** : navigateur (React/TypeScript).
- **Tier 2 – Serveur applicatif** : API Symfony (PHP 8+) exposant une API REST.
- **Tier 3 – Données** : MySQL.

Important : **couches logiques** (controller/service/repository) ≠ **tiers physiques** (conteneurs). On peut déployer plusieurs couches logiques dans un même conteneur tout en conservant la séparation logique dans le code.

Schéma d'architecture (niveau "compréhensible par tous") :

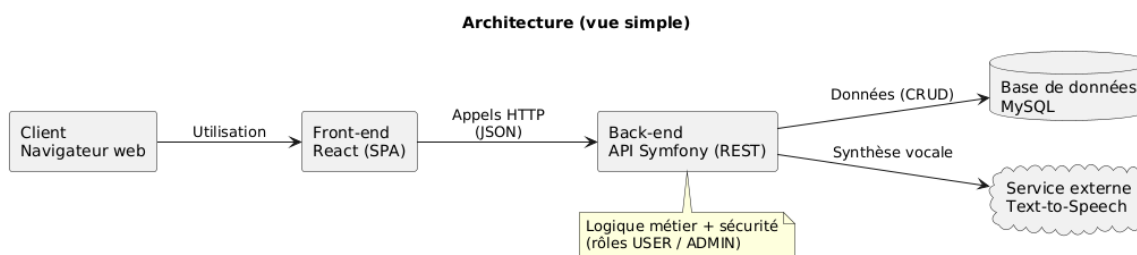


Fig. 6 : Schéma – Architecture 3-tiers

5.3 Séparation des responsabilités et bonnes pratiques

Principes appliqués / prévus :

- **SRP (Single Responsibility)** : services dédiés (ex : ReadingProgressService, FavoriteService).
- **Validation** centralisée (DTO + contraintes Symfony Validator).
- **Gestion des secrets** via .env / variables d'environnement Docker (clé API TTS, secrets JWT, DSN DB).
- **Doctrine** pour éviter l'injection SQL (requêtes paramétrées).
- **DTO / Resource** pour stabiliser le contrat API (ne pas exposer directement toutes les propriétés des entités).
- **Tests** : PHPUnit (unit + intégration API), et côté front Jest/RTL (prévu au jalon 5).

5.4 Composants externes / bibliothèques

Back-end (Symfony) :

- **Doctrine ORM** (entités/repositories/migrations),
- **Symfony Security** (authentification + rôles USER/ADMIN),
- Authentification sécurisée adaptée à une SPA (ex : **JWT** ou **session** selon la stratégie retenue).

Externe :

- **Service de synthèse vocale (Text-to-Speech)** consommé par l'API.
- L'audio n'est **pas stocké en BDD** : génération à la demande à partir de Page.content.

Front-end :

- React + TypeScript,
 - appels HTTP (fetch/axios),
 - gestion d'état selon les besoins du front.
-

6. Schémas complémentaires (optionnel)

6.1 États de publication d'une histoire

Une histoire suit un cycle simple :

- DRAFT : visible uniquement en admin (préparation),
- PUBLISHED : visible dans le catalogue public,
- ARCHIVED : retirée du catalogue (conservée pour historique / audit).

Ce workflow est implémenté par `Story.status` (voir MPD Jalon 3).

7. Conclusion du jalon 4

À l'issue de ce jalon, la conception technique est suffisamment détaillée pour démarrer le développement :

- diagrammes UML complets (use cases, séquences, classes),
- architecture multi-couches claire (API REST + React SPA + MySQL),
- intégration prévue de l'API externe de synthèse vocale.